```
        …
    }
```

Αντίθετα, η ArrayList<Integer> έχει τη μορφή

```
    class ArrayListInteger {
        Integer[] store;
        …
        …
    }
```

Είναι προφανές πως οι δύο κλάσεις δεν συνδέονται με σχέσεις ιεραρχίας. Γενικότερα ισχύει πως αν έχουμε δύο κλάσεις A και B, ο τύπος MyClass<A> δεν σχετίζεται ιεραρχικά με τον τύπο MyClass<B>, ακόμη και αν η κλάση B είναι παράγωγη της A. Ο κοινός πρόγονος των MyClass<A> και MyClass<B> είναι η κλάση Object [1].

Γενικότερα, όταν μια γενίκευση κληρονομεί μία άλλη ή υλοποιεί μια γενικευμένη διεπαφή, η μεταξύ τους σχέση κληρονομικότητας διατηρείται μόνο για την ίδια παράμετρο τύπου. Για παράδειγμα, η ArrayList<E> υλοποιεί την List<E>. Επομένως, ένα ArrayList<Integer> είναι ένα List<Integer>, δεν είναι όμως ένα List<Number>, εξάλλου δεν είναι καν ArrayList<Number>.

## 17.5 Ο χαρακτήρας μπαλαντέρ

Ας υποθέσουμε τώρα πως χρειαζόμαστε μια συνάρτηση που να υπολογίζει το άθροισμα μιας λίστας αριθμών. Η συνάρτηση θα πρέπει να δουλεύει με όλες τις κλάσεις που υλοποιούν τη διεπαφή List αλλά και με όλους τους τύπους αριθμών, δηλαδή με όλες τις παράγωγες κλάσεις της Number. Με άλλα λόγια, θα μας εξυπηρετούσε μία συνάρτηση που μπορεί να λάβει ως παράμετρο και ArrayList<Double> και LinkedList<Integer> αλλά και όλους τους υπόλοιπους συνδυασμούς λιστών και αριθμών.

Τη δυνατότητα αυτή μας παρέχει ο χαρακτήρας μπαλαντέρ (WildCard) που αναπαρίσταται με το αγγλικό ερωτηματικό:

```
static double sum(List<? extends Number> l) {
    double sum = 0d;
    for (int i = 0; i < l.size(); i++) {
        sum += l.get(i).doubleValue();
    }
    return sum;
}
```

***Κώδικας 17.7*** *Αξιοποίηση του χαρακτήρα μπαλαντέρ με άνω όριο*

Η έκφραση List<? extends Number> στη λίστα παραμέτρων της sum σημαίνει πως η συνάρτηση δέχεται ως παραμέτρους αντικείμενο οποιασδήποτε κλάσης υλοποιεί την List, ενώ τα στοιχεία του είναι Number ή παράγωγης της Number. Έτσι, οι κλήσεις στον κώδικα 17.8 είναι όλες σωστές:

```
public static void main(String[] args) {
    List<Double> ld=new ArrayList<>();
    List<Integer> li=new ArrayList<>();
    List<Number> ln=new LinkedList<>();
    for (int i=0; i<10; i++) {
        ld.add((double)i);
        li.add(i);
        ln.add(i);
    }
    System.out.println(sum1(ld));
    System.out.println(sum1(li));
    System.out.println(sum1(ln));
}
```

***Κώδικας 17.8*** *Κλήσεις της γενικευμένης συνάρτησης sum*

Ας σημειωθεί πως εναλλακτική προσέγγιση σε αυτό το πρόβλημα αποτελεί μία συνάρτηση με την ακόλουθη διεπαφή:

```
static <T extends Number> double sum1(List<T> l)
```

Στο παράδειγμα που αναπτύχθηκε στους κώδικες 17.7 και 17.8 περιορίσαμε τα στοιχεία της λίστας ώστε τα στοιχεία της να είναι Number ή παράγωγες κλάσεις. Με άλλα λόγια, δώσαμε το άνω όριο (upper bound) των κλάσεων που είναι αποδεκτές ως τύπος της λίστας. Αυτό ήταν αναγκαίο, καθώς μέσα στο σώμα της sum εκτελούμε προσθέσεις με αυτά τα στοιχεία. Αν όμως θέλαμε μια επεξεργασία που δεν απαιτεί συγκεκριμένο τύπο θα μπορούσαμε να χρησιμοποιήσουμε τον χαρακτήρα μπαλαντέρ χωρίς όρια όπως δείχνει ο κώδικας 17.9:

```
    static void prt(List<?> l) {
        for (int i=0; i<l.size(); i++) {
            System.out.println(l.get(i));
        }
    }
```

***Κώδικας 17.9*** *Αξιοποίηση του χαρακτήρα μπαλαντέρ χωρίς όρια*

Ας υποθέσουμε τώρα πως χρειαζόμαστε μία συνάρτηση που να εισάγει αντικείμενα τύπου Integer σε μία λίστα. Εκ πρώτης όψεως φαίνεται εύκολη απαίτηση, εκτός αν λάβουμε υπόψη ότι σε αντικείμενα Integer μπορούμε να αναφερόμαστε με μεταβλητές τύπου Number αλλά και με μεταβλητές τύπου Object, δηλαδή με μεταβλητές που ο τύπος τους είναι Integer ή κάποιος πρόγονος της Integer. Επομένως, η συνάρτησή μας θα πρέπει να ανταποκρίνεται σωστά με List<Integer>, με List<Number> και με List<Object>.
Μια τέτοια απαίτηση μπορούμε να υλοποιήσουμε με χρήση μπαλαντέρ με κάτω όριο (lower bound).

```
import java.util.ArrayList;
import java.util.List;

public class WCLowBounds {

    static void insert(List<? super Integer> l) {
        Integer i = 3;
        l.add(i);
    }

    public static void main(String[] args) {
        ArrayList<Integer> li = new ArrayList<>();
        ArrayList<Number> ln = new ArrayList<>();
        ArrayList<Object> lo = new ArrayList<>();
        insert(li);
        insert(ln);
        insert(lo);
    }
}
```

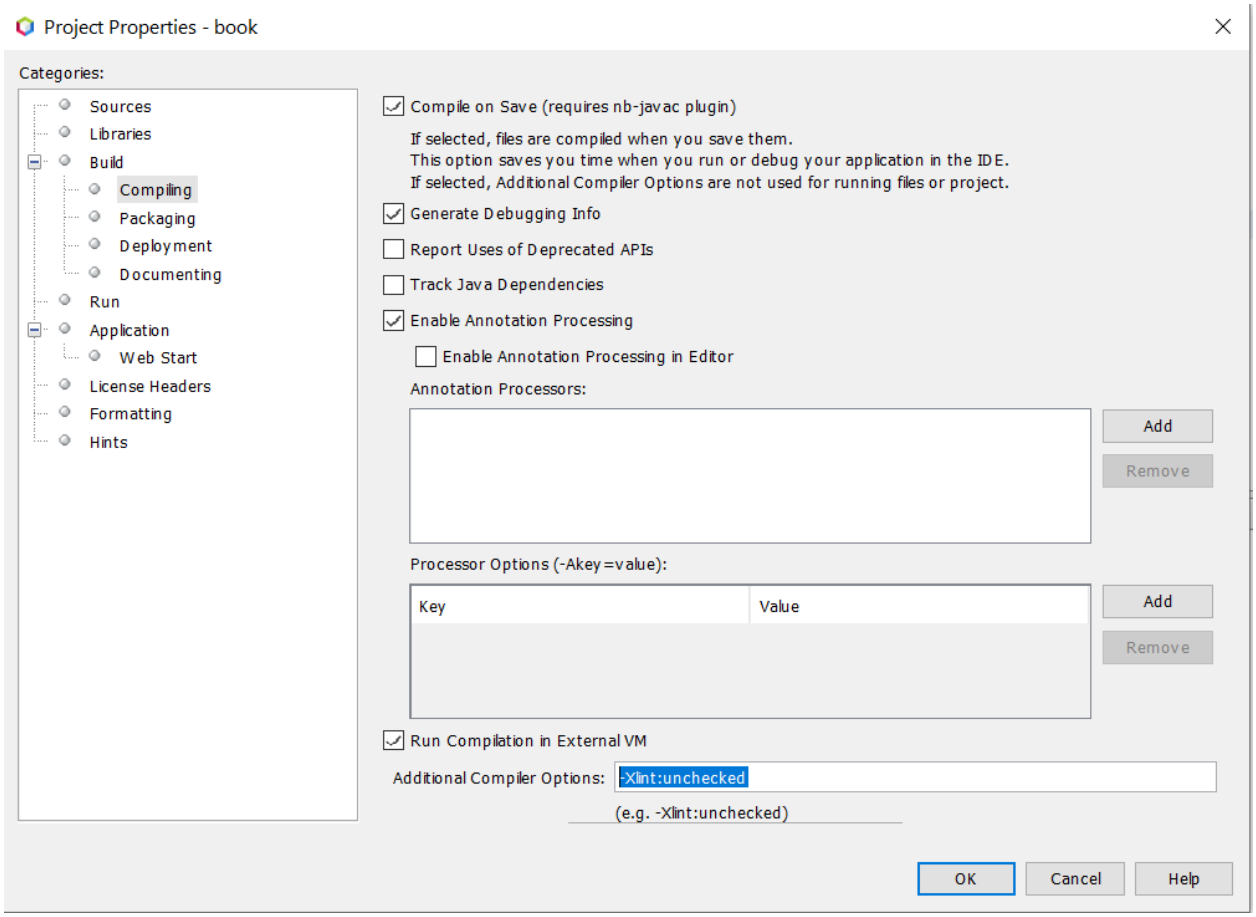
***Κώδικας 17.10*** *Αξιοποίηση του χαρακτήρα μπαλαντέρ με κάτω όριο*

Στον κώδικα 17.10 η συνάρτηση insert δηλώνει την παράμετρό της με χρήση μπαλαντέρ με κάτω όριο. Όπως φαίνεται στον κώδικα, η δήλωση γίνεται με τη δεσμευμένη λέξη super. Η insert μπορεί να λάβει ως παράμετρο οποιαδήποτε λίστα από Integers, Numbers ή Objects. Στη λίστα όμως επιτρέπεται μόνο η εισαγωγή Integer.

Ανακεφαλαιώνοντας, επισημαίνουμε τα εξής: Η List<Number> δεν είναι γονική της List<Integer>. Και οι δύο κλάσεις είναι παράγωγες της List<?>. Όμως η List<? extends Number> είναι γονική της List<? extends Integer>.

## 17.6 Μη Ασφαλείς Λειτουργίες

Σε μερικές περιπτώσεις έχουμε χρησιμοποιήσει μη ασφαλείς λειτουργίες (unsafe operations). Ο μεταγλωττιστής της Java επιτρέπει τις μη ασφαλείς λειτουργίες, καθώς σε πολλές περιπτώσεις είναι αναγκαίες. Εξ ορισμού δεν παράγει καν προειδοποιήσεις. Αν θέλετε να δείτε τις μη ασφαλείς λειτουργίες του κώδικά σας θα πρέπει να μεταγλωττίσετε με την παράμετρο -Xlint:unchecked. Μεταβείτε στα Properties του project, επιλέξτε Compiling και ενημερώστε τις επιπρόσθετες επιλογές μεταγλώττισης (Additional Compiler Options) όπως δείχνει η εικόνα 17.1:




***Εικόνα 17.1*** *Επιπρόσθετες επιλογές μεταγλώττισης*

Τι είναι όμως ακριβώς οι μη ασφαλείς λειτουργίες; Θα το εξηγήσουμε με ένα παράδειγμα. Έστω ότι θέλουμε να υλοποιήσουμε μία γενικευμένη συνάρτηση ταξινόμησης πινάκων χωρίς να χρησιμοποιήσουμε την sort της Arrays. Προκειμένου να ταξινομηθούν τα στοιχεία ενός πίνακα θα πρέπει να είναι συγκρίσιμα μεταξύ τους. Οι κλάσεις που υλοποιούν τη διεπαφή Comparable<T> παρέχουν τη δυνατότητα σύγκρισης των αντικειμένων τους. Πολλές προκαθορισμένες κλάσεις της Java υλοποιούν την Comparable<T>. Αυτή καθορίζει τη φυσική σειρά (natural order) των αντικειμένων. Μπορείτε να βρείτε περισσότερες πληροφορίες στην τεκμηρίωση της Java. Πάντως, η συνάρτησή μας θα πρέπει να λαμβάνει πίνακες από αντικείμενα Comparable, όπως δείχνει ο κώδικας 17.11:

```
    private static <E> void swap(E[] a, int i, int j) {
        if (i != j) {
            E temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }

    public static <E extends Comparable<E>> void sort(E[] a) {
        for (int i = 0; i < a.length - 1; i++) {
            int smallest = i;
```

```
            for (int j = i + 1; j < a.length; j++) {
                if (a[j].compareTo(a[smallest]) <= 0) {
                    smallest = j;
                }
            }
            swap(a, i, smallest);
        }
    }
```

***Κώδικας 17.11*** *Γενικευμένη έκδοση της sort πινάκων*

Ας υποθέσουμε πως έχουμε στη διάθεσή μας και την κλάση Tmp του κώδικα 17.12.

```
class Tmp implements Comparable<Tmp> {

    private final String id;
    private final String description;

    public Tmp(String id, String description) {
        this.id = id;
        this.description = description;
    }

    @Override
    public int compareTo(Tmp o) {
        return id.compareTo(o.id);
    }

    @Override
    public String toString() {
        return id;
    }
}
```

***Κώδικας 17.12*** *Η Tmp που υλοποιεί την διεπαφή Comparable.*

Σε αυτήν την περίπτωση μπορούμε να δημιουργήσουμε πίνακες ως εξής:

```
Integer[] ints = new Integer[10];
Double[] doubles = new Double[10];
Tmp[] tmps = new Tmp[10];
```

Αν δώσουμε τιμές στα στοιχεία σε κάθε έναν από τους πίνακες ints, doubles και tmps και στη συνέχεια καλέσουμε την sort για κάθε πίνακα, θα διαπιστώσουμε ότι όλοι οι πίνακες ταξινομούνται σωστά χωρίς κανένα πρόβλημα. Θεωρήστε τώρα τον πίνακα

```
Comparable[] l=new Comparable[3];
```

που περιλαμβάνει στοιχεία ως εξής:

```
l[0]=10;
l[1]=2d;
l[2]=new Tmp(Integer.toString(1), Integer.toString(2));
```

και την κλήση

```
sort(tmps);
```

Ο κώδικας θα πέσει με java.lang.ClassCastException. Η κλήση sort(tmps) συνιστά μη ασφαλή κλήση, διότι αν προσέξετε τη διεπαφή της sort θα δείτε πως απαιτεί πίνακα στοιχείων τύπου E ο οποίος να είναι Comparable.

Ωστόσο είναι συγκεκριμένος τύπος. Αντίθετα στον πίνακα l περιλαμβάνονται μεν Comparables διαφορετικών ωστόσο τύπων. Έτσι, όταν μέσα στην sort η compareTo επιχειρεί να συγκρίνει ένα αντικείμενο τύπου Integer με ένα τύπου Double ή τύπου Tmp, προσπαθεί να μετατρέψει τον τύπο Double ή Tmp σε Integer, αποτυγχάνει και πέφτει με ClassCastException. Αν ωστόσο όλα τα στοιχεία του πίνακα l αφορούν την ίδια κλάση δεν υπάρχει πρόβλημα παρότι η κλήση συνεχίζει να χαρακτηρίζεται ως μη ασφαλής. Εναπόκειται επομένως στην ευθύνη του κώδικα-πελάτη να καλέσει την sort με σωστό τρόπο.
Σε γενικές γραμμές θα πρέπει να αποφεύγονται οι μη ασφαλείς λειτουργίες. Σε πολλές περιπτώσεις όμως είναι αναγκαίες. Για παράδειγμα, στον κώδικα 15.8, η συνάρτηση loadCars περιέχει τη γραμμή

```
ArrayList<Car> loaded = (ArrayList<Car>) in.readObject();
```

Η readObject επιστρέφει αντικείμενο τύπου Object το οποίο μετατρέπουμε σε ArrayList<Car>. Ο μεταγλωττιστής δεν μπορεί να γνωρίζει αν η μετατροπή αυτή είναι σωστή. Με αυτήν την έννοια, η μετατροπή χαρακτηρίζεται ως μη ασφαλής. Ωστόσο, είναι αναγκαία εφόσον είναι ο μόνος τρόπος για να φορτώσουμε στη μνήμη ένα σειριοποιημένο ArrayList<Car>. Πάλι εναπόκειται στην ευθύνη του κώδικα πελάτη να κάνει τη σωστή μετατροπή, διαφορετικά θα λάβει ClassCastExeption.
Ένα άλλο παράδειγμα υπάρχει στον κώδικα 12.20. Προσέξτε πως στην αρχή της κλάσης MyHashMap υπάρχει η γραμμή

```
private final ArrayList<Person>[] store = new ArrayList[SIZE];
```

Η Java δεν επιτρέπει τη δημιουργία γενικευμένων πινάκων. Ο τύπος του πίνακα store είναι ArrayList<Person>, ωστόσο, ο πραγματικός πίνακας που δημιουργείται είναι ArrayList (raw type). Προσέξτε ότι κατά τη δημιουργία δεν χρησιμοποιήθηκε ο τελεστής διαμαντιού, διαφορετικά η γραμμή θα είχε ως εξής:

```
private final ArrayList<Person>[] store = new
ArrayList<>[SIZE];
```

Ωστόσο, η τελευταία αυτή γραμμή δεν περνάει μεταγλώττιση. Έτσι, όταν χρειαζόμαστε έναν πίνακα στον οποίο θα αποθηκεύουμε γενικεύσεις, στην πράξη δημιουργούμε έναν πίνακα από Object. Καθώς η Object είναι γονική όλων των κλάσεων, είναι και γονική κάθε γενίκευσης. Με άλλα λόγια σε μία αναφορά τύπου Object μπορούμε να εκχωρήσουμε οποιαδήποτε γενίκευση. Επομένως, οι πίνακες στους οποίους αποθηκεύουμε γενικεύσεις μάς εξαναγκάζουν να χρησιμοποιούμε μη ασφαλείς λειτουργίες. Ας σημειωθεί πως η ίδια η προκαθορισμένη κλάση Collections δεν μεταγλωττίζεται χωρίς προειδοποιήσεις (warnings) για μη ασφαλείς λειτουργίες.

## 17.7 Λυμένες Ασκήσεις

### 17.7.1 MyHashMap<K,V>

Να υλοποιηθεί η γενίκευση MyHashMap<K,V> χωρίς να χρησιμοποιηθεί η προκαθορισμένη γενίκευση HashMap<K,V>. Ο εσωτερικός αποθηκευτικός χώρος της MyHashMap θα πρέπει να είναι ένας πίνακας σε κάθε θέση του οποίου θα τοποθετείται ένα ArrayList στο οποίο θα εισάγονται οι συσχετίσεις K, V ανάλογα με τον κωδικό κατακερματισμού τους. Ο εσωτερικός πίνακας θα δημιουργείται με αρχικό μέγεθος 16. Επιπλέον, θα οριστεί ένας συντελεστής φόρτωσης. Όταν οι κατειλημμένες θέσεις του εσωτερικού πίνακα υπερβαίνουν το μέγεθος του πίνακα επί τον συντελεστή φόρτωσης, το μέγεθος του εσωτερικού πίνακα θα πρέπει να διπλασιάζεται και οι συσχετίσεις να επανατοποθετούνται, δηλαδή να γίνεται επανακατακερματισμός (rehashing). Σε ένα MyHashMap δεν επιτρέπεται η εισαγωγή συσχετίσεων με το ίδιο κλειδί. Η MyHashMap θα πρέπει να υποστηρίζει τις ακόλουθες μεθόδους:

```
public void clear()
```

Διαγράφει όλες τις συσχετίσεις από αυτό το αντικείμενο

```
public V put(K key, V value)
```

Εισάγει τη συσχέτιση μεταξύ key και value. Αν η συσχέτιση υπάρχει ήδη, τότε την ενημερώνει και επιστρέφει την τιμή V που βρέθηκε στο MyHashMap. Αν δεν υπάρχει, την εισάγει και επιστρέφει null.

```
public V get(K key)
```

Επιστρέφει την τιμή V που σχετίζεται με αυτό το κλειδί. Αν το κλειδί δεν βρεθεί, επιστρέφει null.

```
public V remove(K key)
```

Διαγράφει από αυτό το MyHashMap τη συσχέτιση με κλειδί key. Αν η συσχέτιση δεν βρεθεί, επιστρέφει null αλλιώς τη συσχέτιση που διεγράφη.

```
public boolean isEmpty()
```

Επιστρέφει true αν καμία συσχέτιση δεν υπάρχει σε αυτό το MyHshMap.

Προσθέστε main στην οποία δημιουργήστε ένα αντικείμενο MyHashMap<String, String>. Εισάγεται μερικές συσχετίσεις. Ρυθμίστε την αρχική χωρητικότητα του εσωτερικού πίνακα και τον συντελεστή φόρτωσης έτσι ώστε με την εισαγωγή των συσχετίσεων να προκληθεί επανακατακερματισμός. Διαβάστε τις συσχετίσεις με την get. Στη συνέχεια, διαγράψτε τις συσχετίσεις μια προς μία με την remove. Μετά από κάθε διαγραφή, καλέστε την isEmpty. Ελέγξτε ώστε όλα τα αποτελέσματα να είναι τα αναμενόμενα.

**Λύση**

```
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;

public class MyHashMap<K, V> {

    public class Entry<K, V> {

        private final K key;
        private V value;

        public Entry(K key, V value) {
            this.key = key;
            this.value = value;
        }

        public K getKey() {
            return key;
        }

        public V getValue() {
            return value;
        }

        public void setValue(V value) {
            this.value = value;
        }

        @Override
        public int hashCode() {
            int hash = 7;
            hash = 97 * hash + Objects.hashCode(this.key);
            return hash;
        }

        @Override
        public boolean equals(Object obj) {
            if (this == obj) {
                return true;
```

```
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
        }
        final Entry<K, V> other = (Entry<K, V>) obj;
        return Objects.equals(this.key, other.key);
    }
}

private int find(K key, List<Entry<K, V>> list) {
    for (int i = 0; i < list.size(); i++) {

        Entry<K, V> entry = list.get(i);
        if (entry.key.equals(key)) {
            return i;
        }
    }
    return -1;
}

private int size = 0;
private static final double LOADFACTOR = 0.75;
private int noOfLists = 0;
private final int INITIALCAPACITY = 3;
private int capacity = INITIALCAPACITY;
private ArrayList<Entry<K,V>>[] store = new ArrayList[INITIALCAPACITY];

public void clear() {
    store = new ArrayList[INITIALCAPACITY];
    size = 0;
    capacity = INITIALCAPACITY;
    noOfLists = 0;
}

private void rehash() {
    int newCapacity = 2 * capacity;
    int newEntries = 0;
    ArrayList<Entry<K,V>>[] local = new ArrayList[newCapacity];
    for (ArrayList<Entry<K,V>> list : store) {

        if (list != null) {
            for (Entry<K,V> entry : list) {
                                int hashCode = entry.key.hashCode();
                int idx = hashCode % newCapacity;
                if (local[idx] == null) {
                    local[idx] = new ArrayList<Entry<K, V>>();
                    newEntries++;
                }
                local[idx].add(entry);
            }
        }
    }
    store = local;
    noOfLists = newEntries;
    capacity = newCapacity;
}

public V put(K key, V value) {
```

```
        if (noOfLists > LOADFACTOR * capacity) {
            rehash();
        }
        int idx = key.hashCode() % store.length;
        if (store[idx] == null) {
            store[idx] = new ArrayList<Entry<K, V>>();
            noOfLists++;
        }
        Entry<K, V> entry = new Entry<>(key, value);
        int location = store[idx].indexOf(entry);
        Entry<K, V> oldEntry;
        if (location < 0) {
            store[idx].add(entry);
            size++;
            return null;
        } else {
            oldEntry = store[idx].get(location);
            store[idx].set(location, entry);
        }
        return oldEntry.getValue();
    }

    public V get(K key) {
        int idx = key.hashCode() % store.length;

        int location = find(key, store[idx]);
        if (location < 0) {
            return null;
        }
        return store[idx].get(location).getValue();
    }

    public V remove(K key) {
        int idx = key.hashCode() % store.length;
        int location = find(key, store[idx]);
        if (location < 0) {
            return null;
        }
        size--;
        V rVal = store[idx].remove(location).getValue();
        if (store[idx].isEmpty()) {
            noOfLists--;
            store[idx] = null;
        }
        return rVal;
    }

    public boolean isEmpty() {
        return size==0;
    }

    public static void main(String[] args) {
        MyHashMap<String, String> hM = new MyHashMap<>();
        hM.put("key1", "value1");
        hM.put("key2", "value2");
        hM.put("key3", "value3");
        hM.put("key4", "value4");
        hM.put("key5", "value5");
        hM.put("key6", "value6");
        hM.put("key7", "value7");
        hM.put("key8", "value8");
```

```
            System.out.println(hM.get("key1"));
            System.out.println(hM.get("key2"));
            System.out.println(hM.get("key3"));
            System.out.println(hM.get("key4"));
            System.out.println(hM.get("key5"));
            System.out.println(hM.get("key6"));
            System.out.println(hM.get("key7"));
            System.out.println(hM.get("key8"));

            System.out.println(hM.remove("key1"));
            System.out.println(hM.isEmpty());
            System.out.println(hM.remove("key2"));
            System.out.println(hM.isEmpty());
            System.out.println(hM.remove("key3"));
            System.out.println(hM.isEmpty());
            System.out.println(hM.remove("key4"));
            System.out.println(hM.isEmpty());
            System.out.println(hM.remove("key5"));
            System.out.println(hM.isEmpty());
            System.out.println(hM.remove("key6"));
            System.out.println(hM.isEmpty());
            System.out.println(hM.remove("key7"));
            System.out.println(hM.isEmpty());
            System.out.println(hM.remove("key8"));
            System.out.println(hM.isEmpty());
        }
    }
```

***Κώδικας 17.13*** *Η κλάση MyHashMap<K,V>*

Η κλάση MyHashMap<K,V> στον κώδικα 17.11 ορίζει την εσωτερική κλάση Entry<K,V>. Τα αντικείμενα Entry<K,V> συνιστούν τις συσχετίσεις που εισάγονται στο MyHashMap. Από τη στιγμή που δημιουργείται μία τέτοια συσχέτιση, το κλειδί της δεν μπορεί να μεταβληθεί. Για αυτόν τον λόγο το μέλος key δηλώθηκε ως final και δεν παρέχεται αντίστοιχος ρυθμιστής. Καθώς στο MyHashMap δεν επιτρέπεται η εισαγωγή συσχετίσεων με το ίδιο κλειδί, η equals θεωρεί συσχετίσεις με το ίδιο κλειδί ως ίσες, οπότε και η hashCode βασίζεται αποκλειστικά στο κλειδί.

Η κλάση διαθέτει τα μέλη δεδομένα, size που διατηρεί τον αριθμό των καταχωρημένων συσχετίσεων, LOADFACTOR που εκφράζει τον συντελεστή φόρτωσης, noOfLists που διατηρεί τον αριθμό των ενεργών λιστών, INITIALCAPACITY που ρυθμίζει την αρχική χωρητικότητα του εσωτερικού πίνακα, capacity που διατηρεί την τρέχουσα χωρητικότητα του εσωτερικού πίνακα και τον εσωτερικό πίνακα store. Προσέξτε πως ο store ως προς τον τύπο είναι πίνακας από ArrayList<Entry<K,V>>. Ωστόσο ο πίνακας που δημιουργείται είναι πρωταρχικός τύπος, καθώς η δημιουργία πινάκων γενικεύσεων δεν υποστηρίζεται. Επομένως, η δημιουργία του store είναι αναγκαστικά μη ασφαλής λειτουργία. Ωστόσο, η κλάση οργανώνεται κατά τρόπο που δεν υποχρεώνει τον χρήστη της σε μη ασφαλείς λειτουργίες. Αντίθετα, όλες οι μη ασφαλείς λειτουργίες είναι εσωτερικές στην κλάση.

Η find είναι μια ιδιωτική συνάρτηση που ψάχνει σε μία λίστα να βρει τη συσχέτιση με κλειδί key. Αν την βρει επιστρέφει τη θέση της στη λίστα, αν όχι επιστρέφει -1.

Η clear δημιουργεί νέο εσωτερικό χώρο και αρχικοποιεί κατάλληλα τις μεταβλητές size, capacity, και noOfLists.

Η put τοποθετεί μία συσχέτιση σε αυτό το MyHashMap. Η πρώτη ενέργεια της put είναι να ελέγξει τον αριθμό των ενεργών λιστών, δηλαδή τον αριθμό των κατειλημμένων θέσεων του πίνακα store. Αν αυτός υπερβαίνει το γινόμενο LOADFACTOR*capacity, τότε καλεί την rehash για να μεγεθύνει τον εσωτερικό αποθηκευτικό χώρο και να επανατοποθετήσει τις συσχετίσεις. Στη συνέχεια εντοπίζει το κλειδί κατακερματισμού της key και υπολογίζει τη θέση του store στην οποία πρέπει να αποθηκευτεί η συσχέτιση. Αν σε εκείνη τη θέση του πίνακα δεν υπάρχει λίστα, τη δημιουργεί και αυξάνει κατάλληλα την noOfLists. Μετά δημιουργεί τη συσχέτιση ως Entry, ελέγχει αν υπάρχει ήδη στην κατάλληλη λίστα και αν υπάρχει την ενημερώνει και επιστρέφει την παλιά τιμή, αλλιώς την εισάγει και επιστρέφει null. Στην περίπτωση που την εισάγει ενημερώνει και τη μεταβλητή size.

Η get επιστρέφει την τιμή που συσχετίζεται με το κλειδί που λαμβάνει ως παράμετρο. Αν δεν υπάρχει συσχέτιση για αυτό το κλειδί, επιστρέφει null.

Η remove διαγράφει τη συσχέτιση στην οποία αναφέρεται το κλειδί που λαμβάνει ως παράμετρο και μειώνει το size κατά 1, εφόσον βρεθεί το κλειδί. Αν με τη διαγραφή της συσχέτισης η αντίστοιχη λίστα αδειάσει, τότε θέτει τη θέση της στον πίνακα στο null και μειώνει την noOfLists κατά 1.

Η isEmpty βασίζεται στη μεταβλητή empty που ενημερώνεται με κάθε εισαγωγή και διαγραφή συσχετίσεων από αυτό το MyHashMap.

Τέλος, η rehash δημιουργεί έναν νέο πίνακα, επανατοποθετεί τα στοιχεία του store στο νέο πίνακα και ανακατευθύνει τη μεταβλητή store ώστε να αναφέρεται στον νέο πίνακα. Ο παλαιότερος πίνακας μένει χωρίς αναφορά και είναι διαθέσιμος για διαγραφή από τον συλλέκτη απορριμμάτων.

### 17.7.2 Γενικευμένη συνάρτηση υπολογισμού αθροίσματος

Να υλοποιηθεί συνάρτηση sum που λαμβάνει ως παράμετρο λίστα ενός οποιουδήποτε αριθμητικού τύπου και επιστρέφει το άθροισμα των στοιχείων της λίστας ως double. Προσθέστε main στην οποία ελέγξτε την sum με ArrayList<Integer> και LinkedList<Double>.

**Λύση**

Η τυπική παράμετρος της συνάρτησης θα πρέπει να είναι του τύπου List<? extends Number>. Με αυτήν την παράμετρο, η συνάρτηση μπορεί να δεχτεί οποιαδήποτε γενίκευση εφόσον υλοποιεί τη διεπαφή List. Επιπλέον, η λίστα μπορεί να εξειδικεύεται ως προς οποιονδήποτε τύπο κληρονομεί την αφηρημένη κλάση Number. Στην περίπτωση που σκεφτήκατε να ορίσετε την παράμετρο ως List<Number>, προσέξτε το εξής: Αυτή η παράμετρος επιτρέπει μόνο λίστες από Number. Βεβαίως μία λίστα από Number μπορεί να περιέχει ως στοιχεία αντικείμενα των παράγωγων κλάσεων. Ωστόσο, δεν είναι δυνατό να περάσουμε στη θέση της μία λίστα από Integer ή μία λίστα από Double.

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;

public class Calculator {

    public static double sum(List<? extends Number> list) {
        double sum = 0;
        Iterator<? extends Number> it = list.iterator();
        while (it.hasNext()) {
            sum += it.next().doubleValue();
        }
        return sum;
    }

    public static double product(List<? extends Number> list) {
        double product = 1;
        Iterator<? extends Number> it = list.iterator();
        while (it.hasNext()) {
            product *= it.next().doubleValue();
        }
        return product;
    }

    public static void main(String[] args) {
        List<Integer> li = new ArrayList<>();
        List<Double> ld = new LinkedList<>();
        List<Number> ln=new ArrayList<>();

        li.add(5);
        li.add(6);
```

```
            ld.add(5.2);
            ld.add(6.5);
            ln.add(5);
            ln.add(5.5);
            System.out.println(sum(li));
            System.out.println(sum(ld));
            System.out.println(sum(ln));
        }
    }
```

***Κώδικας 17.14*** *Η συνάρτηση sum(List<? extends Number>*

Από τη στιγμή που έχουμε προσδιορίσει σωστά τον τύπο της παραμέτρου, η υλοποίηση της συνάρτησης είναι απλή. Χρησιμοποιώντας τον Iterator που παίρνουμε από τη λίστα, διασχίζουμε όλα τα στοιχεία της και προσθέτουμε την πραγματική τιμή τους στον αθροιστή sum τον οποίο και επιστρέφουμε.

### 17.7.3 Κατασκευή πινάκων

Να υλοποιηθεί στατική συνάρτηση mkA με λίστα παραμέτρων μεταβλητού τύπου του παραμετροποιημένου τύπου Τ. Η συνάρτηση επιστρέφει έναν πίνακα τύπου Τ που περιέχει τα αντικείμενα που περνούν ως πραγματικές παράμετροι κατά την κλήση της.

Να προστεθεί ο ακόλουθος κώδικας στην main:

```
public static void main(String[] args) {
        String[] s = mkA("Jack", "John", "Mary");
        Integer[] k = mkA(1, 2, 3, 4, 5);
        for (int i = 0; i < s.length; i++) {
            System.out.println(s[i]);
            System.out.println(k[i]);
        }
    }
```

**Λύση**

Πρόκειται για πολύ απλή συνάρτηση. Το μόνο κρίσιμο σημείο είναι η σωστή ανάπτυξη της διεπαφής της συνάρτησης. Κατά τα λοιπά, η συνάρτηση απλώς επιστρέφει την παράμετρό της η οποία είναι πίνακας τύπου Τ και περιέχει όλες τις πραγματικές παραμέτρους.

```
public static <T> T[] mkA(T... a) {
    return a;
}
```

## 17.8 Ασκήσεις προς Λύση

### 17.8.1 Γενικευμένη δομή LIFO

Να υλοποιηθεί η γενικευμένη κλάση MyStack<E> που αναπαριστά μια δομή LIFO χωρίς να χρησιμοποιηθεί η προκαθορισμένη Stack<E>. Η κλάση πρέπει να υποστηρίζει τις λειτουργίες empty που επιστρέφει true αν αυτό το stack είναι άδειο, την push(E item) που εισάγει το item στην κορυφή της στοίβας και την pop που διαγράφει και επιστρέφει το item από την κορυφή της στοίβας.

### 17.8.2 Γενικευμένη δομή FIFO

Να υλοποιηθεί η γενικευμένη κλάση Queue<E> που αναπαριστά μια δομή FIFO. Η κλάση πρέπει να υποστηρίζει τις λειτουργίες empty που επιστρέφει true αν αυτή η ουρά είναι άδεια, την add(E item) που εισάγει το item στο τέλος της ουράς και την delete που διαγράφει και επιστρέφει το στοιχείο στην αρχή της ουράς.

### 17.8.3 Γενικευμένη συνάρτηση υπολογισμού γινομένου

Να υλοποιηθεί συνάρτηση product που λαμβάνει ως παράμετρο λίστα ενός οποιουδήποτε αριθμητικού τύπου και επιστρέφει το γινόμενο των στοιχείων της λίστας ως double. Προσθέστε main στην οποία ελέγξτε την product με ArrayList<Integer> και LinkedList<Double>.